

Sight Reading Pro

Final Project Report - X/10500 words

Theresa Thoraldson

2026-09-28

Abstract

This report describes the motivation, literature review, design, implementation, and evaluation of Sight Reading Pro, a full-stack machine learning application for sight-reading practice. The template for this project is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal. This project code be found here: <https://github.com/tthoraldson/uol-final-project/>

Contents

Introduction - (720/1000 words)	3
An overview	3
Why Would People Want This Project?	3
Existing/Similar Projects	4
Literature Review (1903/2500 words)	6
Exploring Sight-Reading in Education	6
Pitch Estimation, Tempo Estimation, Audio Evaluation Models	6
Music Generation	7
Other Models	9
Libraries	10
The Research Gaps	11
Design - (X/2000 words)	12
Domain and Project Users	12
Design Overview	12
Design Choices	18
Implementation (X/2500 words)	19
Overview	19
Docker	19
Frontend	20
Main Backend API	20

Model APIs	20
Evaluation - X words	21
Model Evaluation	21
Failed Approaches	21
Successes	22
Limitations	22
Extensions	22
Conclusion - X words	22
In Conclusion...	22
If I were to do it again	22
Further Work	22
Acknowledgements	23
References	23

Introduction - (720/1000 words)

The template I have chosen is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal.

In music, there is a concept called sight-reading. It involves being given a piece of sheet music and trying to play or sing it right away with little or no preparation. According to the National Association for Music Education, the ability to sight-read music has many benefits including increased confidence, stronger foundations in rhythm and pitch, and less stress when it comes to learning new music pieces [6].

An overview

“Sight Reading Pro”, is an application where users can practice and master sight-reading. Users can use any instrument of their choosing, including voice, to practice their sight-reading abilities.

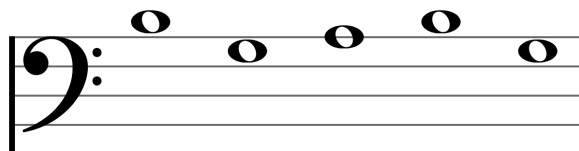


Figure 1: Example sight-reading passage

Sight reading pro is a web application along with multiple APIs, utilizing various state of the art machine learning models to generate music passages for users to practice their sight-reading abilities. The user gets

Users can customize the complexity and difficulty of the generated passages using settings. Users can also select between different styles of passages, such as classical or jazz.

Why Would People Want This Project?

Personally, I’ve always wanted to get better at sight-reading but haven’t found any existing products on the market that make me stick with working through exercises. The generated exercises are often boring, and only target one music area at a time (i.e. advance rhythms, chords, harmony, etc). Having the ability to customize the exercises, and adjusting them to meet my current music learning goals would be amazing.

The existing free projects on the market have limited instrument support, and have no way to track exercise history. The paid options are more focused on music education, instead of individual learning goals. The paid options are also

out of reach to users that want to learn sight-reading, but don't have the budget to do so.

My goals for the project are to make a free, open source, individualized sight-reading platform that can work with any instrument.

Existing/Similar Projects

Sight Reading Factory

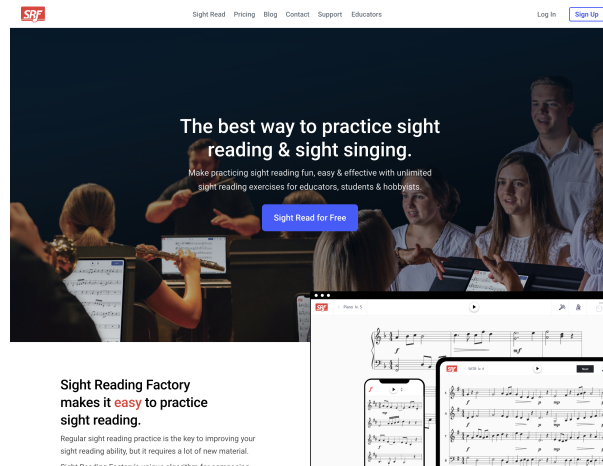


Figure 2: Sight Reading Factory screen capture

Sight Reading Factory[22] is a platform for sight-reading exercises mainly aimed towards educators. This platform supports ~30 instruments, and offers different difficulties of music passages to play. They offer generated music exercises for learners to practice with, getting feedback with each attempt at playing a passage.

Advantages Sight Reading factory has a wide variety of features, including the ability to assess how well a user did on a given passage of music, and the ability to control how difficult generated passages are by limiting the note range, tempo, and rhythmic complexity. Sight Reading Factory supports both general audio/microphone input and MIDI.

Sight reading factory allows you to keep track of what passages you have generated, all of the sight-reading attempts you have made on a given passage, and it offers assessments on new blind passages. This product is the most robust with features that I have found.

Disadvantages I believe the biggest disadvantage to Sight Reading Factory is the fact that it's a subscription, and starts at \$45 per user per year. There's a

free trial, but you have to enter credit card information in order to try it.

Sight Reading factory supports 30 instruments, but has no generic option if your instrument is not supported. You must log in to do anything, which can be just enough friction for learners that want to try sight-reading without committing to a product. This product is also heavily influenced by music educators, and a lot of the features are what you would expect to be in a classroom environment. For example, getting assigned passages to work on.

sightreading.training

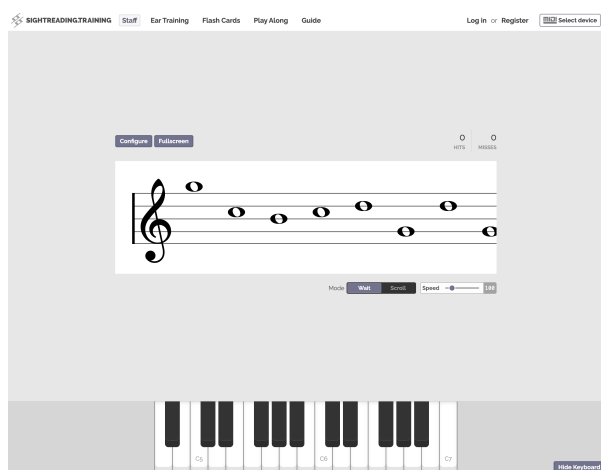


Figure 3: sightreading.training screen capture

sightreading.training[23] is a free and open source sight-reading platform. It offers sight-reading tools, and other tools for learning and playing music.

Advantages sightreading.training supports MIDI, or allows the user to play on a piano that's in the browser. Having a virtual piano is a cool idea, as it makes sight-reading accessible to everyone, including those that don't have an instrument of their own. This website is also free, and you can get started practicing right away. Users have the ability to login and save settings for their sight-reading passages, but not the passages themselves.

Disadvantages sightreading.training only supports MIDI and the virtual piano. If you have an instrument that you can't plug into your computer, you can't use this application. There are no settings to change rhythmic complexity. All of the notes have the same length.

Literature Review (1903/2500 words)

Exploring Sight-Reading in Education

Understanding sight-reading, and how to improve this ability has been an area of study for over 100 years. One research study shows that aural-spatial skills (ear training) and technical proficiency skills were both essential to sight-reading[38].

In more recent times, a review was done on AI-education research related to music education. It concludes that “AI is under-utilized for generation despite its potential for education”[33]. The music generation part of the literature review goes deeper into music generation related to music exercises for general technical improvement.

Pitch Estimation, Tempo Estimation, Audio Evaluation Models

There are lots of existing pre-trained models available that do different tasks related to music. This is an overview of the models I found most relevant to this project, including the models that I ended up using.

CREPE

CREPE is a deep convolutional neural network that does pitch estimation [39]. Given an audio file, such as a .wav or .mp3 file, CREPE will give a predicted frequency in hertz alongside a confidence score for every 10 milliseconds. Having a confidence interval allows for

There is also a demo of CREPE[3] that runs fully in the browser using Tensorflow JS (tfjs)[25]. It shows real time pitch estimation based on microphone input. In regards to this project, being able to have real time pitch estimation would allow for immediate feedback on a music passage.

TempoCNN

TempoCNN[47] is a group of Convolutional Neural Network (CNN) models that estimate a given audio file’s tempo, measured in beats per minute (BPM). These models were trained on datasets that included mainly ballroom dancing music, and electronic dance music. This paper does note that there is a lack of various genres of music, including jazz, classical or reggae, and believes that the models would perform better if they had found and included this kind of data.

For my project I believe TempoCNN is a good candidate model for being able to automatically detect the tempo that the user’s recording was played at, and also use it for giving feedback on a given passage.

Music Generation

Basic Pitch

Basic Pitch[32] is a model built by spotify that takes in an audio file (.wav or .mp3), and returns a MIDI file containing either the single pitch (one instrument/voice) or multi pitch (multiple instruments/voices/notes). Basic Pitch works in the realm of “Automatic Music Transcription”, an area of research used to automatically create symbolic representations of music.

I believe Basic Pitch is a good candidate model that I can use to create backing tracks, and be able to have them stored as midi files. This way the project can take advantage of technologies such as WebMIDI[28] for playback of a generated passage.

MusicGEN

MusicGEN[34] is capable of taking a text based input, and generating a full song. This model can also be prompted with an existing melody and a text prompt, and output a song based on the original melody.

This series of models is promising for offering customized backing tracks for generated sight-reading exercises. The exercises could be tailored to include the correct genre of backing track.

While the music generated is really high quality, the models are also large and require a lot of compute. The smallest model in the MusicGEN model series contains 300 million parameters, and requires ~16gb of GPU RAM to run.

NotaGen

NotaGen[51] is a model that generates classical sheet music. The paper proposes a new “CLaMP-DPO” method for reinforcement learning, which increases musicality when compared to traditional human annotation or predefined rewards. CLaMP 2[55] is a multi-modal music information retriever that was trained across 101 languages. NotaGen uses CLaMP 2 in its reinforcement learning CLaMPD-PO, where it takes the CLaMP model to do iterative optimization, and then DPO[41].

This model requires at least 8GB of GPU RAM to run the smallest model. One goal of “Sight Reader Pro” is to have low response times for any generative process. Because of the large compute requirements for NotaGen, I don’t believe it will be a good fit for the project.

Text To Music

text-to-music[20] is a model that takes in a prompt, and produces ABC notation[1] from that prompt. It’s a BART[41] based model that has been finetuned with a dataset similar to wikimusic[21]. I find this to be one of the more compelling models, as the prompt examples contain the same kind of natural language that I’m looking to use in Sight Reader Pro:

Example training prompts for text-to-music, directly from model card on hugging face:

- This is a traditional Irish dance music. Note Length-1/8
Meter-6/8 Key-D
- This is a jazz-swing lead sheet with chord and vocal.

I find the ability to use natural language to prompt for specific time signatures and music styles to fit perfectly with that I'm going to build with Sight Reader Pro. I also like that the output is in ABC format, which is just plain text. This makes it easy to store, and easy to process.

The training dataset used also contained the genre of music for each tune it was trained on, which is an important feature to have for when I generate prompts for creating new music.

Chat Musician

Chat Musician[57] is a 4 billion paramter model based on LLAMA2[50] that takes natural language prompts, and can do three tasks: Music Theory QA, Composing (using ABC notation[1]), and chatting. They created their own dataset, which they call *Music Pile*, that includes a variety of information outside of music including Wikipedia[30], Irishman[53], and synthetic music data that they created using Chat-GPT 4[45]. The model performs extremely well on music understanding, music reasoning and music knowlege tasks. The paper talks about music generation as a way to compress music data into the ABC format for better compression, but doesn't talk about music generation as it's own goal.

Text2Midi

Text2Midi[26] is an LLM that generates midi files from text descriptions. It takes LLM Embeddings from the text description, then uses a Transformer decoder to generate tokes that are then converted to a midi file.

MIDI[9] is a standard interface to define communication between instruments and a computer. MIDI can be live, like using a keyboard as a MIDI input, or MIDI can be a file format that's used to define the pitch, rhythm and tone of an instrument or an ensemble.

Using MIDI as a way to generate music seems like an interesting approach. There seems to be a handful of libraries that can handle MIDI to ABC notation and vice versa, but that doesn't seem to be the norm. I still want to experiment with generative MIDI models, as they have the potential to generate different music from the text to ABC style models listed above.

MidiLLM

Similar to Text2Midi, MidiLLM[54] is an LLM that produces MIDI. Text2Midi is based on Llama 3[37]. It uses a two stage training process so that the model can understand text related to music, and also MIDI. MidiLLM outperforms Text2Midi on the TheoryTab dataset[35], which specializes in multitrack music generation. This model is compatible with the HuggingFace Transformers library[52].

Considering my application is for sight-reading, which is a solo task, I don't believe using a model that specializes in creating MIDI for ensembles .

AudioGen

AudioGen[40] is an auto-regressive generative model that creates audio samples based on text prompts. It was trained on a variety of different . The public model found on HuggingFace[7] has 1.5 billion parameters.

One of my original ideas for Sight Reader Pro was to have the ability to create backing tracks for the generated sight-reading exercises. While I could also generate midi for this task, having more realistic audio that sounds like a real guitar, or a reach orchestra seems appealing.

My main worry with this model, similar to MusicGen[34], is that it's too large, and too computationally expensive to use alongside simple, ABC notation based sight reading exercises. It would be cool to incorporate something like this into a base set of exercises on Sight Reading Pro in the future, where the ABC and backing track are already generated, but this will not work for on the fly sight-reading exercise generation.

Other Models

Qwen, Qwen ONNX

Qwen3[56] is an open source LLM, and is a popular alternative to both Chat GPT and Claude. It can

On HuggingFace, there's an ONNX Runtime[15] implementation of Qwen3[16], having 0.6 billion parameters. ONNX Runtime can run models both on mobile devices and directly in the browser. Having a cross-platform compatible model for any conversation or retrieval tasks could be nice! And using something like TensorflowJS[25] or TransformersJS[8] to run Qwen 3 directly in the browser is appealing.

I believe this would be more of a stretch feature than anything I would include in this iteration of Sight Reader Pro.

Llama

Llama 2[50] and Llama 3[37] are a series of LLMs put out by Meta. As listed above, they're commonly used as base models for building fine tuned and task specific models. The Llama series is great at typical LLM tasks like conversation and retrieval (Just like Qwen!).

Similar to Qwen 3, using a Llama model directly is a stretch feature.

Libraries

Librosa

Librosa[44] is a python library that has lots of audio processing functions. It offers pre-made models for tasks like onset detection, rhythm features, pitch and tuning, onset detection, and much more.

I plan on using librosa to validate the results of some of the larger models I'll be using. Librosa can handle beat tracking and tempo related tasks, and it also has an implementation of Pyin[42], a frequency estimator does the same task as CREPE[4], as mentioned above.

music21

"music21[29] is a Python-based toolkit for computer-aided musicology." It has tons of helper methods to convert between common types (abc -> midi), It has has a large corpus of various free music in different formats for testing. I believe this library will be helpful for quickly converting between music formats, and maybe finding some testing data.

muspy

muspy[[36]][12] is library mainly focused on creating music analysis related pipelines, and facilitating music i/o between other popular music formats and libraries. It supports both ABC notation[1] and MIDI, along with many more. It also has it's own set of muspy specific classes, focused on symbolic music representation.

This library will be helpful if I want to explore storing the sight-reading exercises in a more symbolic way, and it could also be helpful for it's music i/o functions

mir_eval

mir_eval[[46]][10] describes itself as " Python library which provides a transparent, standardized, and straightforward way to evaluate Music Information Retrieval systems." It has tons of methods specifically for Music Information Retrieval, including tempo validation, key detection, and segmentation.

This library will be extremely useful in helping to evaluate how well my selected models perform, and to calculate metrics for users sight-reading exercise attempts.

The Research Gaps

While there are many *text-to-music* machine learning models that satisfy generating new sight-reading exercises, there is a lack of applications that put it to use in real time. There is also a lack of using the AMT style tools and models to produce feedback on sight reading exercises. The two main solutions (Sight Reading Factory[22] and sightreading.training[23]) don't have the ability to customize exercises based on genre, or any prompt for that matter.

Design - (X/2000 words)

As stated in the intro, the template I am choosing is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal.

Domain and Project Users

The goal of this project was to create a sight-reading web application that utilizes various machine learning models to create unique sight-reading exercises. The *target user* for this project is an independent musician that's looking to improve their sight-reading abilities, and is interested in a customized experience based on their instrument of choice, skill level and music genre of choice. The *domain* of this project could be considered to be music education, but I'm aiming for it to be specifically independent music education.

This project is not ideal for users just learning an instrument, as this application requires at least an elementary understanding of reading sheet music, and playing their instrument of choice.

In my initial feedback survey on the mockups, there was almost an even split of people who were musicians, and people who were not. I believe this is the best kind of feedback, as I can iterate on my features to work for everyone who may want to try sight-reading, not just the experts.

Would you describe yourself as a musician, either professionally or as a hobby?

7 responses

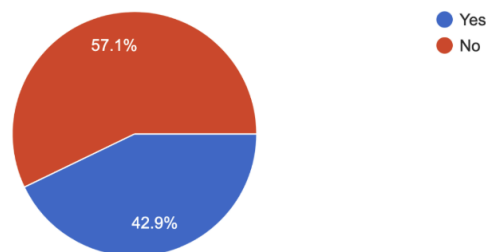


Figure 4: Initial Design Survey: Musician Question

I also asked if users were familiar with the concept of sight-reading. Surprisingly, the majority did! I believe this validates the name, *Sight Reader Pro*, as something that could be recognizable.

Design Overview

The designs below are both my initial mockups for what Sight Reader Pro might look like, and the final version.

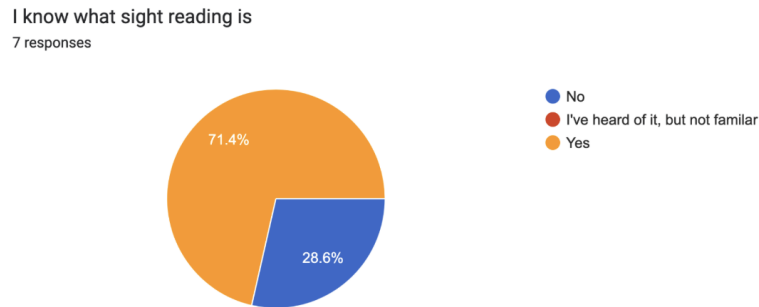


Figure 5: Initial Design Survey: Sight Reading Question

Home Page

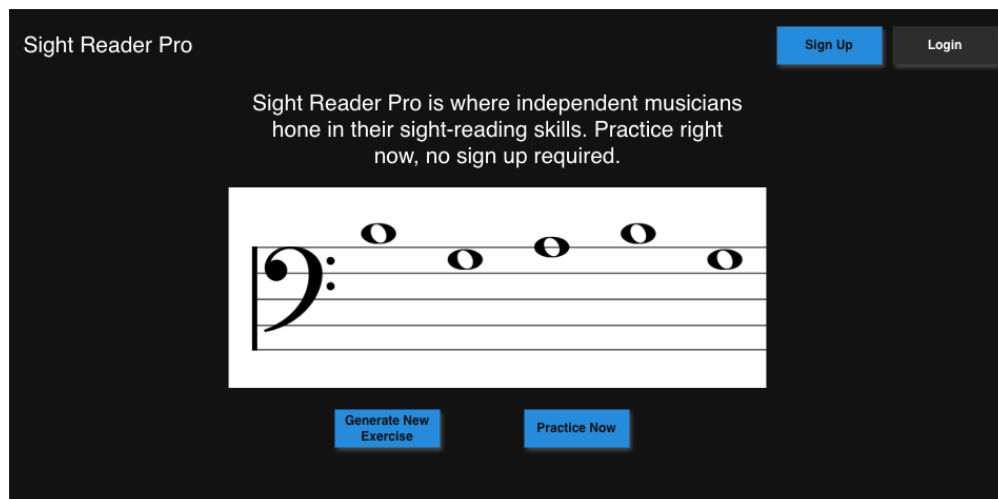


Figure 6: Sight Reading Pro Home Initial Design Mockup

In the initial mockup phase, I was planning on implementing a home page that had the majority of the functionality of the exercise page, but with some marketing flare for first time visitors. After I got into implementing the app, I decided to ax creating a different home page. The exercise page is the only page, and all of the functionality of the final Sight Reading Pro happens there.

Before Recording Exercise Attempt

The mockup exercise page before an attempt is very clean, only including two options: connect to an audio source and record an attempt. If the user selects the record, the record button will switch to a stop button. After the stop button is

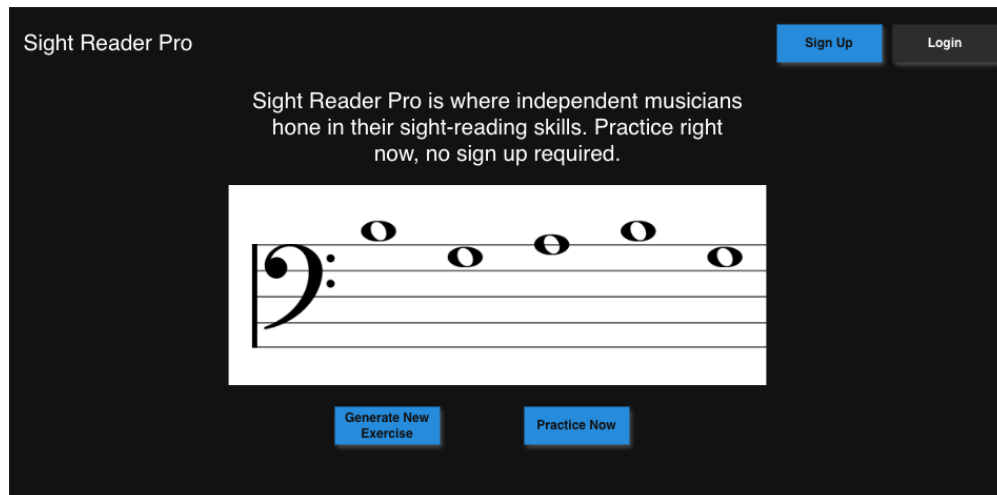


Figure 7: Sight Reading Pro: Home Initial Design Mockup

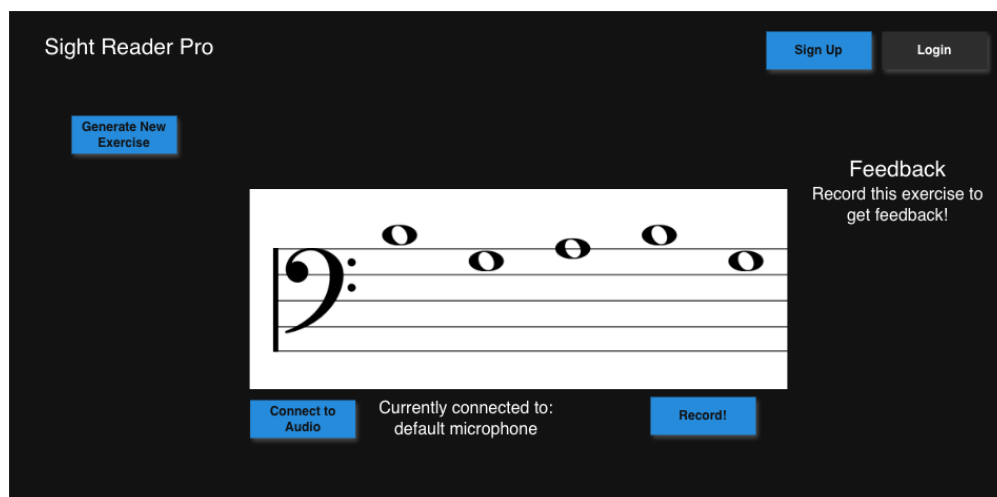


Figure 8: Sight Reading Pro Exercise Before Attempt

selected, the screen will update to the “after recording exercise attempt” screen. In this mockup, there was no option to upload audio.

This is the main exercise page in the application Sight Reading Pro. It's where musicians record their sight reading attempts. Are the options of this page easy to understand?

7 responses

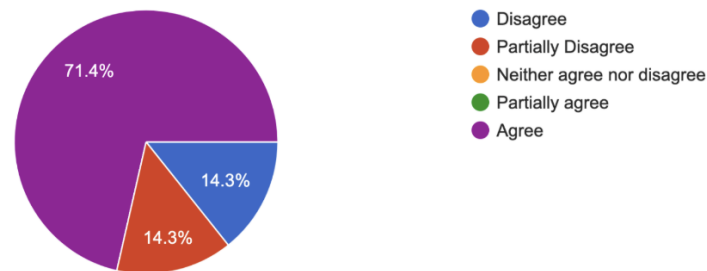


Figure 9: Initial Design Survey: Exercise Before Attempt Feedback

User Feedback The majority of the users surveyed agreed that the options of this page were easy to understand. There was no freeform feedback on this screen.

Final Design The final design ended up adding a few new things. There is a midi playback, that's part of the ABCJS[2] package, and creates a playback of what the exercise sounds like. Considering how easy this feature was to add (It was two lines in the frontend), I thought it would be worth adding in.

The second new feature on this view is the ability to add ABC notation manually. This feature became important for full-stack testing towards the end of creating this project. The music that was being generated was too difficult for me to play, and I wanted to be able to test the same exercise repeatedly after the state of the application had changed.

After Recording Exercise Attempt

This screen is extremely similar to the before exercise attempt screen, but with feedback added from the previous attempt. More feedback will be included based on settings selected, including tempo, rhythm accuracy, etc.

Generate New Exercises

In the mockup I have two settings to choose from, a difficulty slider and a genre selector. I envisioned many more settings, but only ended up adding support to select from a list of common instruments.

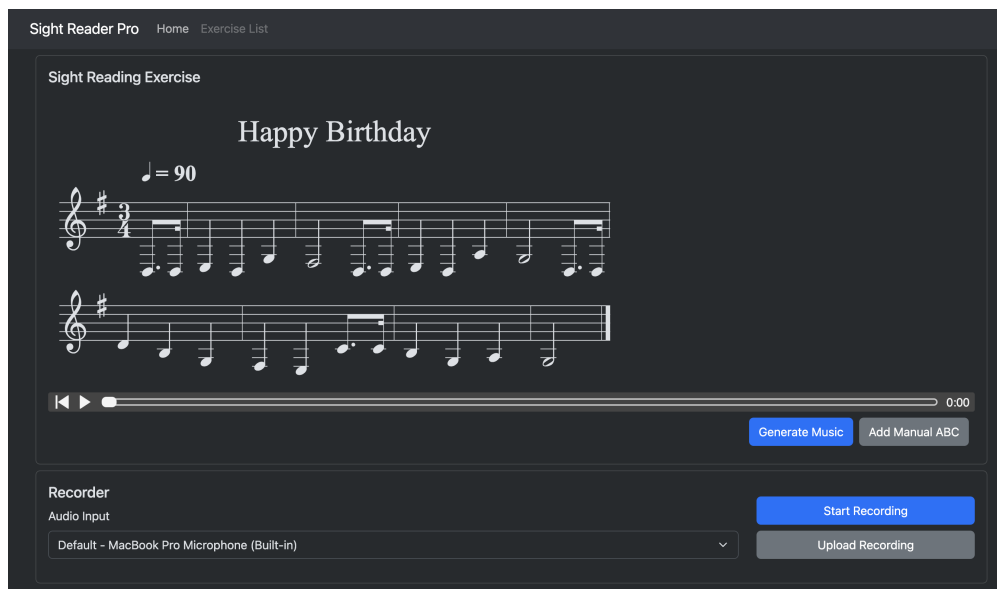


Figure 10: Sight Reading Pro Exercise Before Attempt, Final

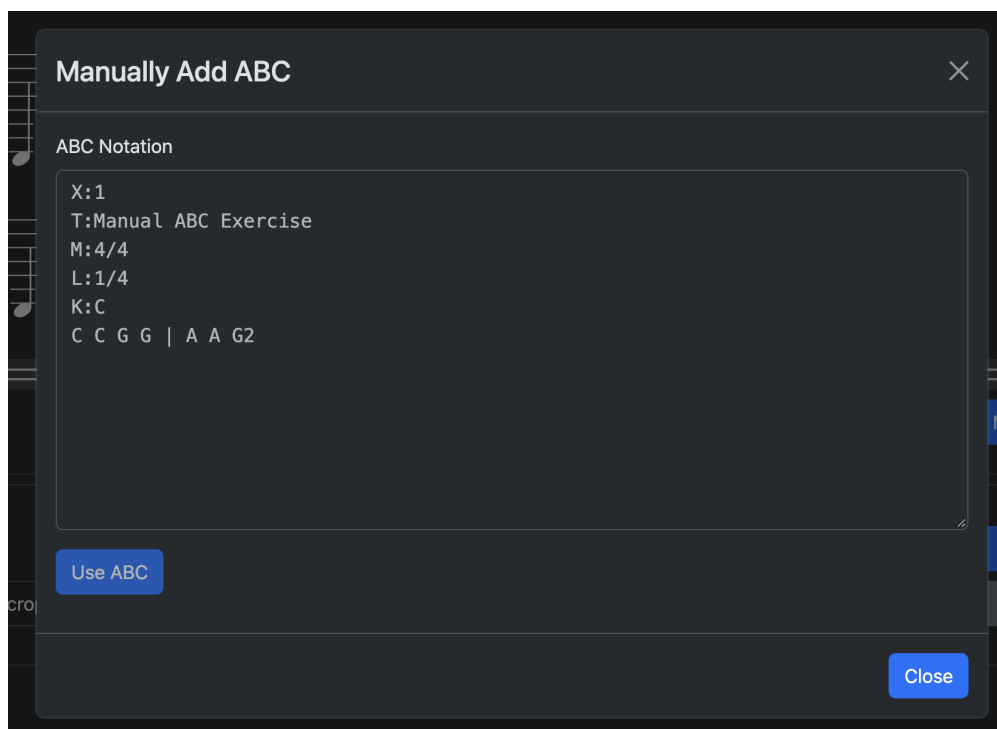


Figure 11: Sight Reading Pro: Add Manual ABC

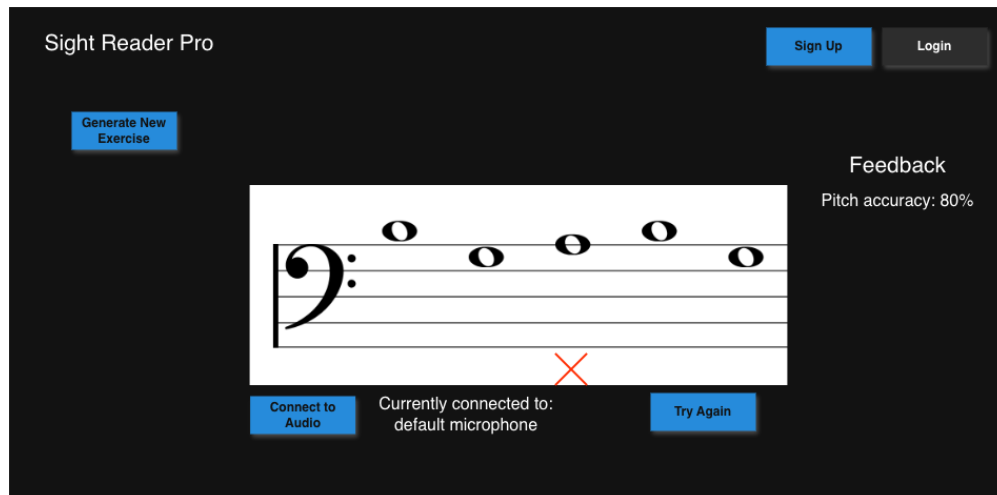


Figure 12: Sight Reading Pro Mockup: Exercise Practice After Attempt

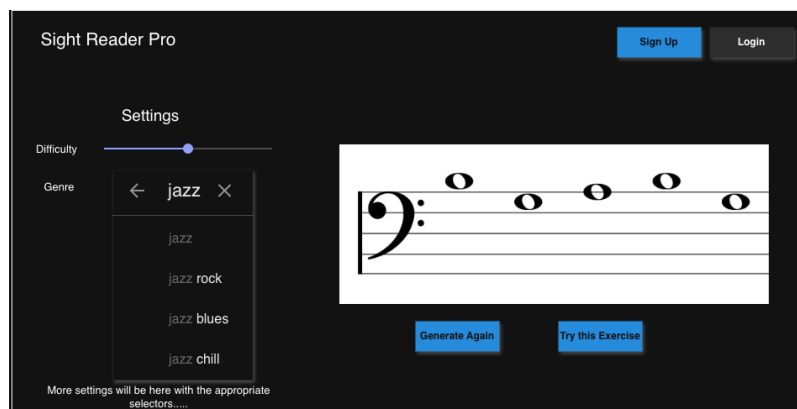


Figure 13: Sight Reading Pro Mockup: New Exercise

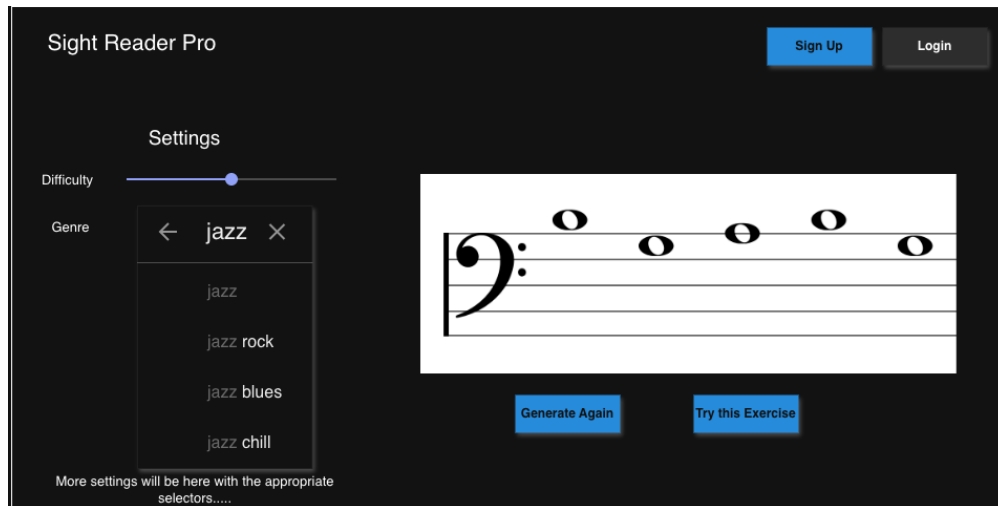


Figure 14: Sight Reading Pro Mockup: New Exercise

Design Choices

Keep it simple

Using lots of

Good experience without

My main goal is to reduce friction in users trying out Sight Reader Pro. All of the core functionality will be in the exercise page if a user is logged in or if they're anonymous.

Customizable Exercise Settings

The settings to generate sight-reading exercises will allow users to generate exercises that are specific to them. For example, I only would want to see bass clef exercises that have a jazzy theme. Each user will be able to “choose their own adventure” for what kind of sight-reading experience they want.

Implementation (X/2500 words)

Overview

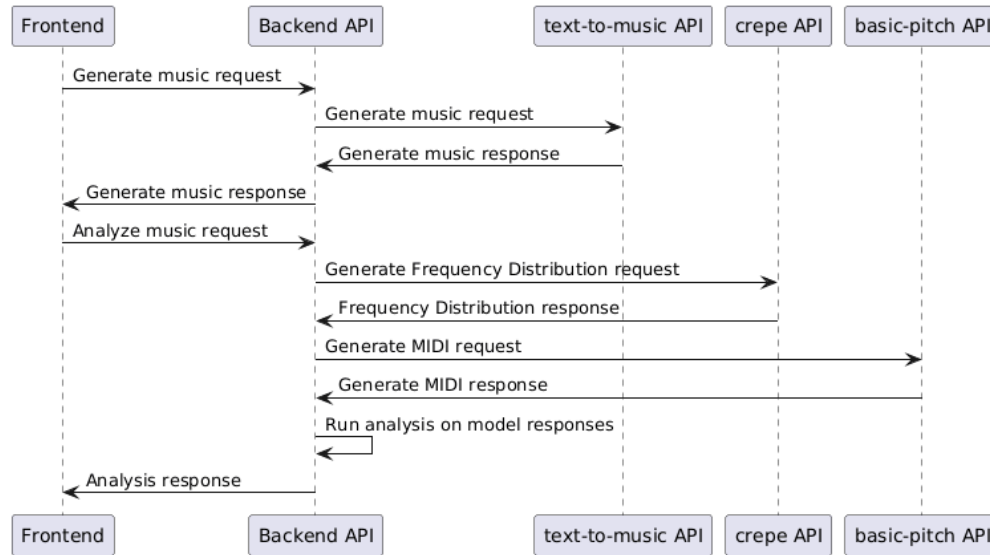


Figure 15: Core Functionality API Calls

Docker

The whole project is wrapped in docker[5]. The frontend, the main API, all of the models, and MLFlow[11] are all in their own containers.

There are two defined networks: `frontend-network` and `model-network`. `frontend-network` has the frontend container and the Main API container. The goal of this network was to isolate what could talk to the Main API, and the Frontend. I didn't want any of the model containers being able to communicate to the frontend, and I didn't want the frontend or any external requests to be able to communicate with the model containers, or the MLFlow container.

Using docker allowed me to better control the dependencies for each model. For example, the `text-to-music model` uses python 3.12, whereas `basic-pitch model` uses python 3.10. These projects would not be able to run in the same container, as they both require different versions of Numpy[13].

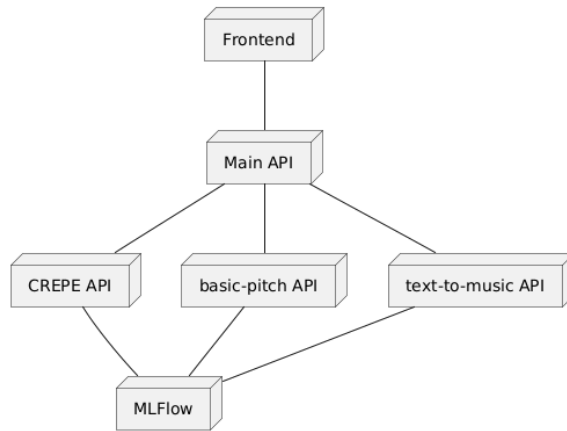


Figure 16: Docker Architecture

Frontend

Libraries Used

The frontend was written using React[17] and uses Vite[27] for the docker/production build. I was already familiar with React, as I use it at my day job, and thought it would be the easiest way for me to make the frontend quickly.. The main styling and core components were built using React Bootstrap[18]. I used abcjs[2] for rendering abc notation, and for

Main Backend API

Model APIs

All of the model containers follow the same basic flow, using a version of python that meets the main package requirements, FastAPI for calling the model, and MLFlow for logging each experiment.

Basic-Pitch API

Text-To-Music API

CREPE API

User feedback

Evaluation - X words

Model Evaluation

One of the core parts of choosing this project template was model evaluation. My main goals for choosing models were the following:

- Does this model have fast inference?
- Can the model generate a variety of music styles?
- Can the model

MusicGen

- Kinda sucked. Took Forever.

NotaGen

- Also kind of sucked.

Text2Midi

- This model took forever.
- The model took ~2 hours to generate anything worth while.

Text To Music

- Loved the results of this model
- Example code found on Huggingface hub/repo worked

Failed Approaches

Porting TextToMusic to ONNX Runtime

Earlier on in this project, I was playing around with the idea of using Transformers.js[8] to grab all of the models for this project, and have inference done using ONNX Runtime[14]. This would allow me to run model inference on the client side, which would be so cool! The Transformers.js documentation made it seem like it would be easy enough, so I gave it a shot. The main python notebook (located at `project/model-exploration/text_to_music_onnx_conversion.ipynb` in the project repo[49]) use to convert Text-To-Music to an onnx format seemed like it was going well, just running into typical input/output shape issues. I made it

as far as uploading it to HuggingFace[48] before I realized running inference on this model required a custom python library, *Samplings*[19]. For a few hours I tried to re-implement both the *TopPSampling* and *TemperatureSampling* methods before I decided to just use what the original author of the model had used, and go back to doing a full python implementation.

Successes

Basic-Pitch works great!

CREPE + Basic-Pitch make a mean combo

Limitations

Extensions

Conclusion - X words

In Conclusion...

If I were to do it again

If I were to start this project over, I believe my approach would change considerably. I think I probably would've stuck with using something like gradio[31] or streamlit[24] to quickly make the frontend would've cut down my workload for getting a testable user interface, and allow me to put more time into prompt tuning the models I chose, and maybe even fine tuning models that would work better to generate sight-reading passages.

There were a lot of really cool audio processing libraries that I wanted to try, such as Librosa[43], but didn't get around to because of my approach. I believe that there are lots of signal processing implementations for audio analysis that I could've utilized if I wasn't so focused on using machine learning models for every aspect of this project.

Further Work

Acknowledgements

- I'm grateful to my sister Anna, and my partner J for proof reading through this report too many times to count, and supporting me during stressful times.
- Pandoc was used to generate this report from a markdown file
- Zotero was used to manage sources, and generate a BibTex file for references/citations
- draw.io was used for all of the Sight Reader Pro mockups
- <https://plantuml.com/> was used for creating my architecture diagrams

References

- [1] Abc notation. <https://abcnotation.com/>.
- [2] Abcjs: Javascript Music Library. <https://abcjs.net>.
- [3] CREPE Interactive Demo. <https://marl.github.io/crepe/>.
- [4] Crepe: CREPE pitch tracker.
- [5] Docker | Secure Sandboxes for AI Agents.
- [6] 2024. Empowering Students through Sight Reading. <https://nafme.org/>.
- [7] 2023. Facebook/audiogen-medium · Hugging Face. <https://huggingface.co/facebook/audiogen-medium>.
- [8] 2026. Huggingface/transformers.js. Hugging Face.
- [9] 2026. MIDI. *Wikipedia*. (Sep. 2026).
- [10] Mir_eval Documentation — mir_eval 0.8.2 documentation. <https://mir-eval.readthedocs.io/latest/index.html>.
- [11] MLflow - Open Source AI Platform for Agents, LLMs & Models. *MLflow AI Platform*. <https://mlflow.org>.
- [12] Muspy — MusPy documentation. <https://hermandong.com/muspy/doc/muspy.html>.
- [13] NumPy. <https://numpy.org/>.
- [14] ONNX Runtime. <https://onnxruntime.ai/>.
- [15] ONNX Runtime | Home. <https://onnxruntime.ai/>.
- [16] Onnx-community/Qwen3-0.6B-ONNX · Hugging Face. <https://huggingface.co/onnx-community/Qwen3-0.6B-ONNX>.
- [17] React. <https://react.dev/>.
- [18] React Bootstrap | React Bootstrap. <https://react-bootstrap.netlify.app/>.
- [19] 2022. Samplings. *PyPI*. <https://pypi.org/project/samplings/>.
- [20] 2022. Sander-wood/text-to-music · Hugging Face. <https://huggingface.co/sander-wood/text-to-music>.
- [21] 2026. Sander-wood/wikimusictext · Datasets at Hugging Face. <https://huggingface.co/datasets/sander-wood/wikimusictext>.
- [22] Sight Reading Factory®. <https://www.sightreadingfactory.com>.
- [23] Sight Reading Trainer. <https://sightreading.training/>.
- [24] 2021. Streamlit • A faster way to build and share data apps. <https://streamlit.io/>.

- [25] TensorFlow.js | Machine Learning for JavaScript Developers. *TensorFlow*. <https://www.tensorflow.org/js>.
- [26] Text2midi: Generating Symbolic Music from Captions. <https://arxiv.org/html/2412.16526v1>.
- [27] Vite. *vitejs*. <https://vite.dev>.
- [28] 2026. Web MIDI API | MDN. *MDN Web Docs*. https://developer.mozilla.org/en-US/docs/Web/API/Web_MIDI_API.
- [29] What is Music21? — Music21 Documentation. <https://music21.org/music21docs/about/what.html>.
- [30] 2025. Wikimedia/wikipedia · Datasets at Hugging Face. <https://huggingface.co/datasets/wikimedia/wikipedia>.
- [31] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. and Zou, J. 2019. Gradio: Hassle-free sharing and testing of ML models in the wild.
- [32] Bittner, R.M., Bosch, J.J., Rubinstein, D., Meseguer-Brocal, G. and Ewert, S. 2022. A Lightweight Instrument-Agnostic Model for Polyphonic Note Transcription and Multipitch Estimation. *arXiv*.
- [33] Carnovalini, F., Espírito Santo, L. and Wiggins, G.A. 2025. Personalized Music Education: A Systematic Review of AI Generation Methods. *IEEE Access*. 13, (2025), 207433–207447. <https://doi.org/10.1109/ACCESS.2025.3639898>.
- [34] Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y. and Défossez, A. 2024. Simple and Controllable Music Generation. *arXiv*.
- [35] Donahue, C., Thickstun, J. and Liang, P. 2022. Melody transcription via generative pre-training. *arXiv.org*. <https://arxiv.org/abs/2212.01884v1>.
- [36] Dong, H.-W., Chen, K., McAuley, J. and Berg-Kirkpatrick, T. 2020. MusPy: A Toolkit for Symbolic Music Generation. *arXiv*.
- [37] Grattafiori, A. et al. 2024. The Llama 3 Herd of Models. *arXiv*.
- [38] Hayward, C.M. and Gromko, J.E. 2009. Relationships among Music Sight-Reading and Technical Proficiency, Spatial Visualization, and Aural Discrimination. *Journal of Research in Music Education*. 57, 1 (2009), 26–36.
- [39] Kim, J.W., Salamon, J., Li, P. and Bello, J.P. 2018. CREPE: A Convolutional Representation for Pitch Estimation. *arXiv*.
- [40] Kreuk, F., Synnaeve, G., Polyak, A., Singer, U., Défossez, A., Copet, J., Parikh, D., Taigman, Y. and Adi, Y. 2023. AudioGen: Textually Guided Audio Generation. *arXiv*.
- [41] Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V. and Zettlemoyer, L. 2019. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *arXiv*.

- [42] Mauch, M. and Dixon, S. 2014. PYIN: A fundamental frequency estimator using probabilistic threshold distributions. *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (Florence, Italy, May 2014), 659–663.
- [43] McFee, B., Raffel, C., Liang, D., Ellis, D., McVicar, M., Battenberg, E. and Nieto, O. 2015. Librosa: Audio and Music Signal Analysis in Python. *Python in Science Conference* (Austin, Texas, 2015), 18–24.
- [44] McFee, B., Raffel, C., Liang, D., Ellis, D.P.W., McVicar, M., Battenberg, E. and Nieto, O. 2015. Librosa: Audio and Music Signal Analysis in Python. *SciPy 2015*. (Jul. 2015). <https://doi.org/10.25080/Majora-7b98e3ed-003>.
- [45] OpenAI et al. 2024. GPT-4 Technical Report. arXiv.
- [46] Raffel, C., McFee, B., Humphrey, E.J., Salamon, J., Nieto, O., Liang, D. and Ellis, D.P.W. A TRANSPARENT IMPLEMENTATION OF COMMON MIR METRICS.
- [47] Schreiber, H. and Müller, M. 2018. A Single-step Approach to Musical Tempo Estimation using a Convolutional Neural Network. *Proceedings of the 19th International Society for Music Information Retrieval Conference*. (Sep. 2018), 98–105. <https://doi.org/10.5281/zenodo.1492353>.
- [48] Thoraldson, T. Tthoraldson/text-to-music-onnxruntime · Hugging Face. <https://huggingface.co/tthoraldson/text-to-music-onnxruntime>.
- [49] Thoraldson, T. 2026. Tthoraldson/uol-final-project.
- [50] Touvron, H. et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv.
- [51] Wang, Y., Wu, S., Hu, J., Du, X., Peng, Y., Huang, Y., Fan, S., Li, X., Yu, F. and Sun, M. 2025. NotaGen: Advancing Musicality in Symbolic Music Generation with Large Language Model Training Paradigms. arXiv.
- [52] Wolf, T. et al. 2020. Transformers: State-of-the-Art Natural Language Processing. Association for Computational Linguistics.
- [53] Wu, S., Li, X., Yu, F. and Sun, M. TunesFormer: Forming Irish Tunes with Control Codes by Bar Patching.
- [54] Wu, S.-L., Kim, Y. and Huang, C.-Z.A. 2025. MIDI-LLM: Adapting Large Language Models for Text-to-MIDI Music Generation. arXiv.
- [55] Wu, S., Wang, Y., Guo, R.Y.Z., Tan, X., Zhang, G., Zhou, M., Chen, J., Mu, X., Gao, Y., Dong, Y., Liu, J., Li, X., Yu, F. and Sun, M. CLaMP 2: Multimodal Music Information Retrieval Across 101 Languages Using Large Language Models.
- [56] Yang, A. et al. 2025. Qwen3 Technical Report. arXiv.
- [57] Yuan, R. et al. 2024. ChatMusician: Understanding and Generating Music Intrinsically with LLM. arXiv.